

TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

LABORATORY WORK NO. 1

CREATIONAL DESIGN PATTERNS IN DOMI (C#)

Student: Lucian-Adrian Gavril

Group: FAF-241

Coordinator: asist. univ., Daniel Istrati

Chișinău, 2026

TABLE OF CONTENTS

ABSTRACT 5

LIST OF ABBREVIATIONS 6

INTRODUCTION 7

1 THEORY: WHY CREATIONAL PATTERNS EXIST 8

1.1 The problem with `new` everywhere 8

1.2 The six patterns and the question each one answers 8

1.3 Class-level and object-level patterns 9

1.4 OOP terms used in this report 9

2 THE DOMI DOMAIN 11

2.1 One place for the rules 11

2.2 Joining a group 12

3 BUILDER 13

3.1 The idea in plain words 13

3.2 Problem it solves 13

3.3 Where it fits in Domi 13

3.4 Code 14

3.5 How it works 14

3.6 Why not something simpler 16

3.7 Disadvantages 16

4 ABSTRACT FACTORY 17

4.1 The idea in plain words 17

4.2 Problem 17

4.3 Where it fits in Domi 17

4.4 Code 17

4.5 How it works 18

4.6 Why not something simpler 19

4.7 Trade-offs 20

5 PROTOTYPE 21

5.1 The idea in plain words 21

5.2 Problem 21

5.3 Where it fits in Domi 21

5.4 Code 21

5.5 How it works 22
5.6 Shallow copy and deep copy 22
5.7 Trade-offs 25
6 SINGLETON 26
6.1 First, what `static` means 26
6.2 The idea in plain words 26
6.3 Problem it solves 26
6.4 Where it fits in Domi 26
6.5 Where Singleton is used in real software 27
6.6 Where Singleton should NOT be used 27
6.7 Code 27
6.8 How it works 27
6.9 Why thread safety matters 28
6.10 Disadvantages: why Singleton is often criticised 29
7 FACTORY METHOD 30
7.1 The idea in plain words 30
7.2 Problem, step by step 30
7.3 Where it fits in Domi 30
7.4 Code 30
7.5 How it works 31
7.6 Factory Method and Abstract Factory compared 33
7.7 Trade-offs 33
8 OBJECT POOL 34
8.1 The idea in plain words 34
8.2 Problem 34
8.3 Where it fits in Domi 34
8.4 Code 34
8.5 How it works 35
8.6 The two rules of pooling 37
8.7 Trade-offs 37
9 HOW THE PATTERNS WORK TOGETHER 38
9.1 Connection to SOLID (Lab 0) 38
9.2 Choosing a pattern 39
10 RESULTS 40

10.1 Project structure 41
CONCLUSIONS 42
REFERENCES 43

ABSTRACT

This laboratory work applies the six creational design patterns to Domi, a service where people form groups to rent a home together. Builder assembles and validates a group step by step. Abstract Factory produces a group and a suggested renter that always match. Prototype copies a group's settings into a new group without its identity or members. Singleton holds the shared catalog defaults, Factory Method lets subclasses choose the group inside a fixed workflow, and Object Pool reuses buffers for group summaries. For each pattern the report explains the problem it solves, where it fits in Domi, the code, why a simpler approach is not enough, and its disadvantages, and connects the patterns to the SOLID principles from Laboratory work No. 0.

Keywords: creational patterns, builder, abstract factory, prototype, singleton, factory method, object pool

LIST OF ABBREVIATIONS

DI	dependency injection
GoF	Gang of Four (Gamma, Helm, Johnson, Vlissides)
ID	identifier
JSON	JavaScript Object Notation
SOLID	single responsibility, open/closed, Liskov substitution, interface segregation, dependency inversion
UML	Unified Modeling Language

INTRODUCTION

The task is to study the creational design patterns, implement at least three of them in a project of our choice, and be able to explain the remaining ones. The report must state the project theme, the patterns used, short descriptions with code, and a UML diagram.

The deeper goal is to learn when object creation itself becomes a design problem. Writing `new` is the simplest way to create an object, and for most objects it is enough. The creational patterns answer the cases where it is not enough: when the caller should not know the concrete class, when construction is complicated, when objects must match each other, when a copy is easier than a construction, when only one instance may exist, or when creating an object is too expensive to repeat.

The project is **Domi**, a service that helps people who do not know each other form a group to rent a home together. Domi is a good fit because creating its objects has real rules: a group must have a valid cost and capacity, a suggested renter must be able to afford the group, and a copied group must not inherit the original's identity or members.

Implemented in the main demo: **Builder**, **Abstract Factory**, **Prototype**. Implemented as additional examples: **Singleton**, **Factory Method**, **Object Pool**.

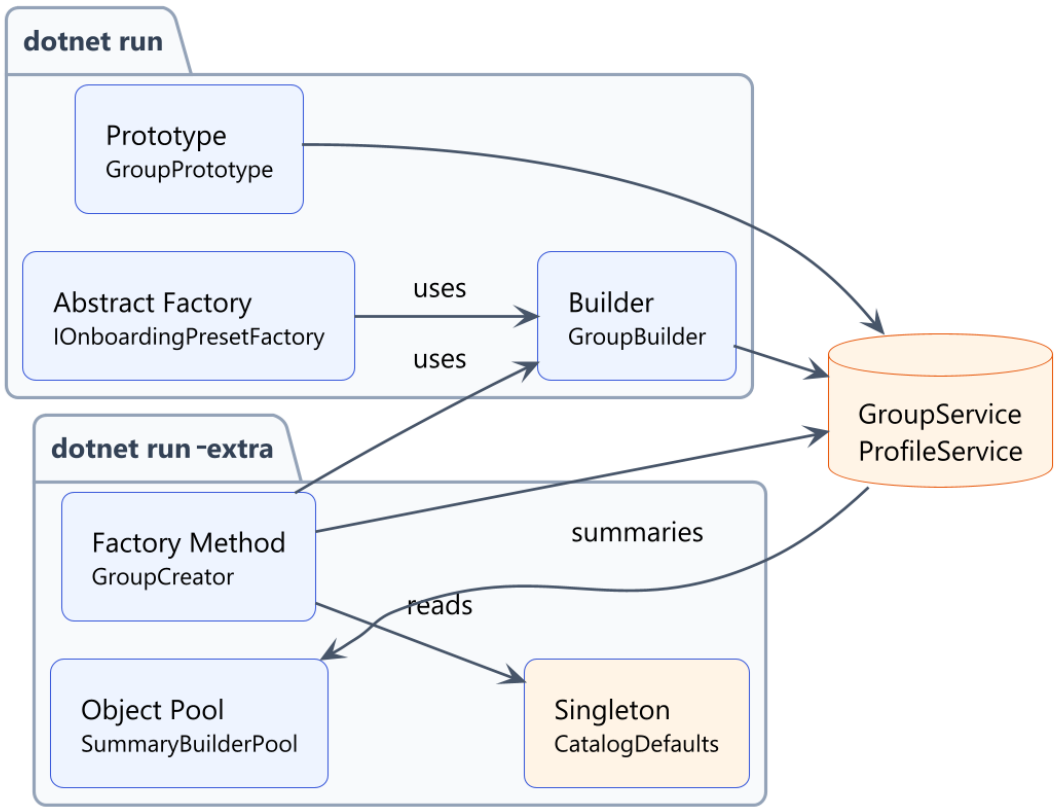


Figure 1 – The six patterns in the project and how they relate. Arrows show which pattern uses another.

1 THEORY: WHY CREATIONAL PATTERNS EXIST

1.1 The problem with `new` everywhere

Consider a service that creates groups directly:

```
public GroupView CreateBudgetGroup()
{
    var group = new Group("group-7", new GroupInput("Budget home", 450, 4));
    ...
}
```

This works, but three things are now fixed inside the service: the concrete class, the values, and the validation order. When Domi adds a student group, the service must be edited. That is a violation of the **Open/Closed Principle** from Lab 0 (a module should be open for extension and closed for modification). The service also depends on a concrete class instead of an abstraction, which breaks **Dependency Inversion**. If several services repeat the same construction steps, the rules drift apart, which breaks **Single Responsibility**.

Creational patterns fix this by moving the act of creation into one place that the rest of the code calls through an abstraction or a controlled entry point. The code that *uses* an object stops deciding *how* it is made.

1.2 The six patterns and the question each one answers

Pattern	The creation problem it solves	One-line idea
Singleton	Something must exist exactly once	Private constructor, one static instance
Factory Method	Same workflow, different object at one step	Subclass overrides the method that creates
Abstract Factory	Objects must come in matching families	One factory object creates every member of a family
Builder	Construction has many steps or settings that must be valid together	Set values step by step, validate in `Build()`
Prototype	A new object should start as a copy of an existing one	The object knows how to clone itself correctly
Object Pool	Creating the object is expensive or limited	Reuse released objects instead of creating new ones

The first five come from *Design Patterns: Elements of Reusable Object-Oriented Software* (Gamma, Helm, Johnson, Vlissides, 1994) [1]. Object Pool is not in that book but is commonly taught with them because it also controls how instances come into existence.

1.3 Class-level and object-level patterns

The patterns differ in the OOP mechanism they rely on, and this is the clearest way to tell Factory Method from Abstract Factory:

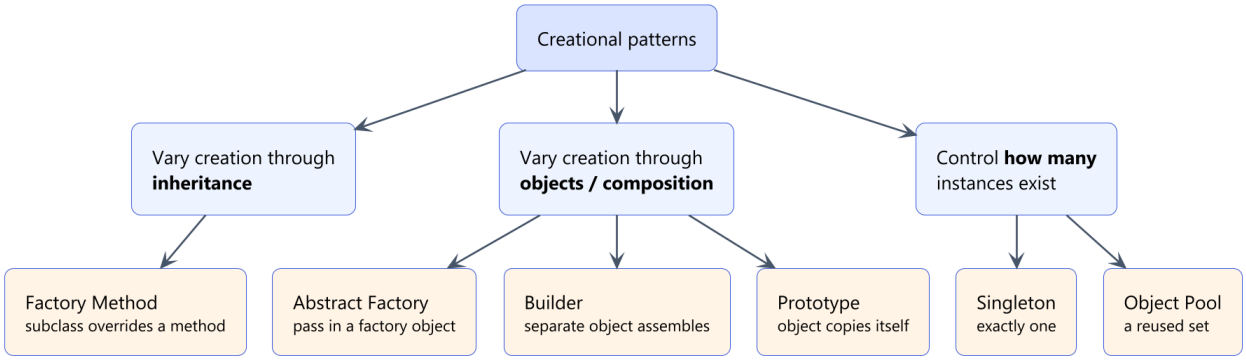


Figure 1.1 – Factory Method varies the created object through inheritance. Abstract Factory, Builder and Prototype vary it through objects. Singleton and Object Pool control the number of instances.

1.4 OOP terms used in this report

- **Class / object.** `Group` is a class (a blueprint). `group-2` is an object (one concrete group in memory);
- **Interface.** A contract of methods without implementation. `IOnboardingPresetFactory` says ”you can create a group draft and a profile,” without saying how;
- **Inheritance.** A child class gets all the parent's methods and can add or change some. `StudentGroupCre : GroupCreator`;
- **Abstract class and method.** `GroupCreator` is abstract: it cannot be created with `new` and exists only to be inherited from. Its `abstract` method has no body, and every child must `override` it;
- **Reference.** A variable of class type holds an address, not the object. `Group a = b;` gives two names for one group. This matters for Prototype;
- **`static`.** Belongs to the class, not to an object, so there is one copy for the whole program. 100 groups have 100 names but one static field. It does not mean ”cannot change”. `CatalogDefaults.Instance` is static. This matters for Singleton;
- **`private`.** Only code inside the same class can see or change it. `_memberProfileIds` is private so outside code must go through `AddMember`, which checks the rules;
- **`override`.** A child class writes its own version of a method the parent declared `abstract` or `virtual`. When the parent calls that method, the child's version runs. This is **polymorphism**. This matters for Factory Method;
- **`record`.** A C# class meant to hold data, with value equality and `with` for making a changed copy.

`GroupInput` and `ProfileInput` are records;

2 THE DOMI DOMAIN

Domi has two core objects. A **Group** has a name, a maximum monthly cost per person, a capacity from 1 to 20, and a list of member IDs. A **Profile** has a name and a private monthly budget. A profile can join a group only when its budget covers the cost and the group has a free place.

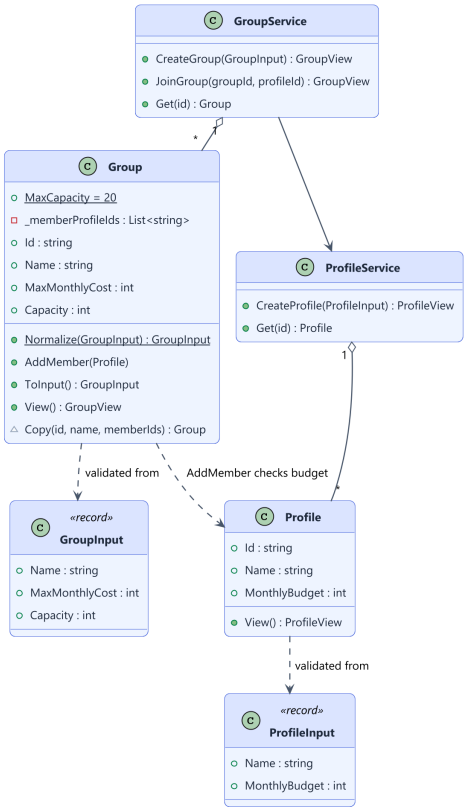


Figure 2.1 – Domain model. Inputs are what a caller sends; the services turn them into stored objects with IDs.

2.1 One place for the rules

Every pattern in this lab eventually produces a `GroupInput`, and every `GroupInput` passes through one method:

```
public static GroupInput Normalize(GroupInput input)
{
    string name = (input.Name ?? "").Trim();
    if (name == "") throw new DomainException("group name is required");
    if (input.MaxMonthlyCost <= 0) throw new DomainException("maxMonthlyCost must be
   greater than zero");
    if (input.Capacity <= 0) throw new DomainException("capacity must be greater than
     zero");
    if (input.Capacity > MaxCapacity) throw new DomainException($"capacity cannot
   exceed {MaxCapacity} people");
    return input with { Name = name };
}
```

How it works. It trims the name, rejects an empty name, a non-positive cost and a capacity outside 1 to 20, and returns a cleaned copy using `with`. On a rule violation it throws `DomainException`.

Why it matters for this lab. The Builder calls it in `Build()`, the `Group` constructor calls it, and the Prototype calls it through that constructor. Because creation is centralised, no pattern can produce a group the rest of the system would reject.

2.2 Joining a group

```
public void AddMember(Profile profile)
{
    if (_memberProfileIds.Contains(profile.Id)) return;
    if (profile.MonthlyBudget < MaxMonthlyCost) throw new DomainException("monthly
   budget is below the group's cost");
    if (_memberProfileIds.Count >= Capacity) throw new DomainException("group is full
   ");
    _memberProfileIds.Add(profile.Id);
}
```

How it works. A repeated join returns without error, so retrying a request is safe. Then it checks the budget and the capacity *before* changing anything, so a failed join leaves the group unchanged. Only after both checks does it add the ID.

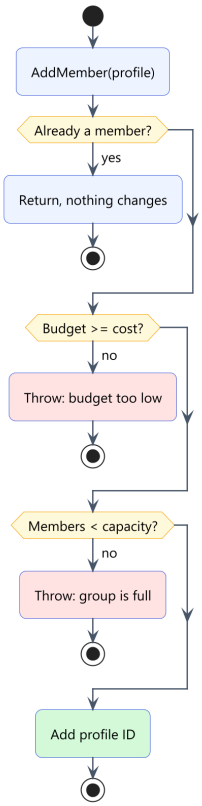


Figure 2.2 – The join rule. Every check happens before the member list changes.

3 BUILDER

3.1 The idea in plain words

Think of ordering at a sandwich shop. You don't shout everything in one breath. You say ”white bread”, then ”chicken”, then ”no onions”, then ”that's it, make it”. The worker collects your choices one by one and only makes the sandwich at the end. If you forgot to choose bread, they stop you at the end, before making anything.

A Builder works the same way. You give it values one at a time with methods that say what each value is (`WithName`, `WithCapacity`), and at the end you call `Build()`. Only then does it check everything and make the object.

3.2 Problem it solves

Problem 1: mixed-up values. A group needs a name, a cost and a capacity. With a normal constructor you pass them in order:

```
new GroupInput("Flat", 450, 4);   // name, cost, capacity
```

Cost and capacity are both `int`. If someone swaps them, `new GroupInput("Flat", 4, 6)`, it still compiles and passes validation, because 4 and 6 are both allowed values. Domi now has a group that costs 4 per month for 6 people, and nobody gets an error. With a Builder the call says `.WithMaxMonthlyCost(450).WithCapacity(4)`, so a swap is visible when you read it.

Problem 2: too many constructors. Domi will grow: city, pets allowed, smoking, move-in date. Some groups set them and some don't. With constructors you end up writing one for every combination:

```
new GroupInput(name, cost, capacity)
new GroupInput(name, cost, capacity, city)
new GroupInput(name, cost, capacity, city, pets)
new GroupInput(name, cost, capacity, city, pets, smoking, moveIn)
```

This growing list is called the **telescoping constructor** problem. With a Builder you call only the `With...` methods you need, and the rest keep their defaults.

Problem 3: checking values together. Some rules involve several values at once. Checking them in the middle, after only the name is set, would fail because the cost isn't set yet. The Builder waits until `Build()`, when everything is there, and checks it all at once.

3.3 Where it fits in Domi

The ”create group” form collects fields one at a time, and the user presses *Create* at the end. The Builder has the same shape: set values in any order, validate once at the end. Other real uses: `StringBuilder` in C# (append text piece by piece, then `ToString()`), building an SQL query or an HTTP request with optional parts, and `WebApplication.CreateBuilder()` in ASP.NET.

3.4 Code

```
public sealed class GroupBuilder
{
    private string _name = "";
    private int _maxMonthlyCost;
    private int _capacity;

    public GroupBuilder WithName(string name)
    {
        _name = name;
        return this;
    }

    public GroupBuilder WithMaxMonthlyCost(int amount)
    {
        _maxMonthlyCost = amount;
        return this;
    }

    public GroupBuilder WithCapacity(int people)
    {
        _capacity = people;
        return this;
    }

    public GroupInput Build() => Group.Normalize(new GroupInput(_name,
    _maxMonthlyCost, _capacity));
}
```

Usage in `Program.cs`:

```
GroupInput builderInput = new GroupBuilder()
    .WithName("Builder example home")
    .WithMaxMonthlyCost(650)
    .WithCapacity(3)
    .Build();
GroupView builderGroup = groups.CreateGroup(builderInput);
```

3.5 How it works

Each `With...` method stores one value in a private field and returns `this`, the same builder object. Because the return type is `GroupBuilder`, the next call can be chained directly, which is called a **fluent interface**. No validation runs while values are being set, so the order of calls does not matter and an incomplete builder is never mistaken for a finished group. `Build()` creates the record and passes it to `Group.Normalize`, which validates all three values together and either returns a clean input or throws.

The Builder returns a `GroupInput`, not a saved `Group`. Saving and assigning an ID is the service's job. The Builder only knows how to assemble a valid request, which keeps its responsibility small.

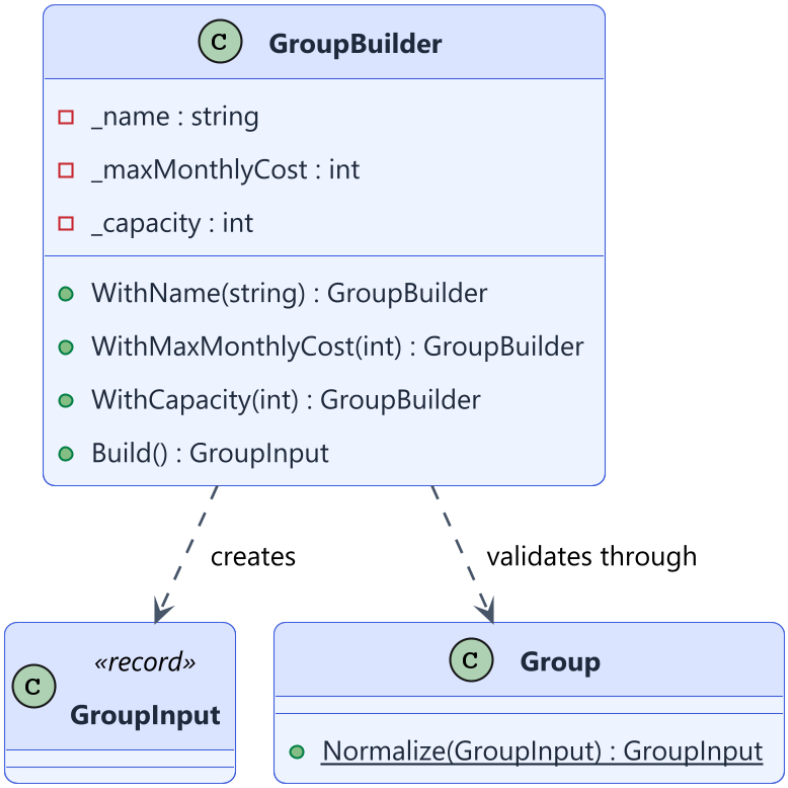


Figure 3.1 – Builder class diagram.

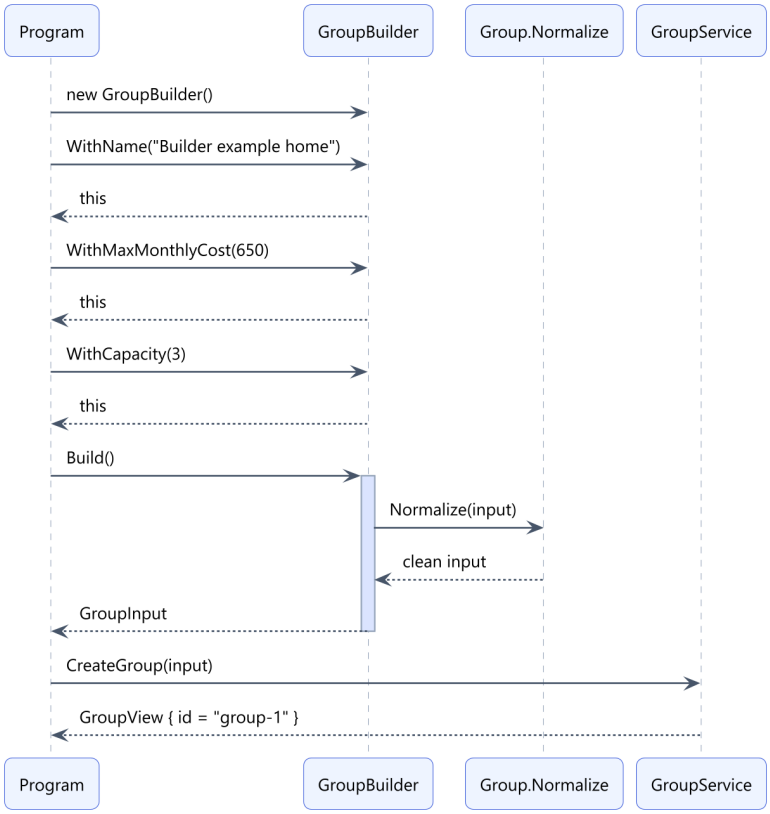


Figure 3.2 – Builder sequence. Validation happens once, in `Build()`.

3.6 Why not something simpler

C# named arguments, `new GroupInput(Name: "Flat", MaxMonthlyCost: 450, Capacity: 4)`, already fix the mixed-up values problem. A Builder is worth its extra class when values arrive in separate steps (a multi-page form, defaults that some callers override) or when validation must see all values together. Domi's onboarding factories use the Builder with partly fixed values, which is the second case.

3.7 Disadvantages

A pattern always costs something. For the Builder:

1. **More code.** For every class you build, you write a second class (`GroupBuilder`) with a field and a method for each value. Add a field to `Group` and you must remember to add it to the Builder too;
2. **Overkill for small objects.** For an object with two values, `new Point(3, 5)` is clearer than `new PointBuilder().WithX(3).WithY(5).Build()`. The Builder only pays off when there are many values or optional ones;
3. **Errors appear later.** A wrong value is reported at `Build()`, not on the line where you set it, so the error points to the end of the chain, not to the mistake;
4. **Forgotten values are not caught by the compiler.** If you forget `.WithCapacity(...)`, the code still compiles, and the problem only shows up when the program runs and `Build()` throws. A constructor with a required parameter would refuse to compile;
5. **Reusing a builder keeps old values.** Call `Build()` twice on the same builder and the second object starts with the first one's values. Start each object with `new GroupBuilder()`;

When to use it: many values, optional values, or rules that check values together. **When not to:** two or three required values, where a constructor or named arguments are enough.

4 ABSTRACT FACTORY

4.1 The idea in plain words

Think of furniture styles in a store. If you pick the ”Modern” style, you get a modern chair *and* a modern table. You never get a modern chair with a vintage table, because one choice of style gives you the whole matching set.

An Abstract Factory is that style. It is an **interface** listing everything in the set (”create a group”, ”create a renter”), and each class that implements it creates one matching family. The code that uses it receives one factory and asks it for every part, so the parts always match.

4.2 Problem

Domi's onboarding suggests a group and a renter profile together. They only make sense as a pair: a budget renter with 500 per month cannot join a comfort group at 700. If the code chooses the group and the profile in two separate places, nothing prevents a mismatched pair, and the first join fails.

4.3 Where it fits in Domi

The onboarding screen asks once: *Budget* or *Comfort*. That single choice must give a matching group and profile. Each style is a **family** of related objects.

4.4 Code

```
public sealed record OnboardingPreset(GroupInput Group, ProfileInput SuggestedProfile
    );

public interface IOnboardingPresetFactory
{
    GroupInput CreateGroupDraft();
    ProfileInput CreateSuggestedProfile();
}

public sealed class BudgetPresetFactory : IOnboardingPresetFactory
{
    public GroupInput CreateGroupDraft() => new GroupBuilder()
        .WithName("Budget shared home")
        .WithMaxMonthlyCost(450)
        .WithCapacity(4)
        .Build();

    public ProfileInput CreateSuggestedProfile() => Presets.ProfileSuggestion("Budget
     renter", 500);
}

public sealed class ComfortPresetFactory : IOnboardingPresetFactory
{
    public GroupInput CreateGroupDraft() => new GroupBuilder()
        .WithName("Comfort shared home")
        .WithMaxMonthlyCost(700)
```

```
        .WithCapacity(2)
        .Build();

    public ProfileInput CreateSuggestedProfile() => Presets.ProfileSuggestion("
    Comfort renter", 900);
}
```

The client code:

```
public static OnboardingPreset Build(IOnboardingPresetFactory factory)
{
    GroupInput group = factory.CreateGroupDraft();
    ProfileInput profile = factory.CreateSuggestedProfile();

    if (profile.MonthlyBudget < group.MaxMonthlyCost)
        throw new DomainException("monthly budget is below the group's cost");

    return new OnboardingPreset(group, profile);
}
```

4.5 How it works

The interface declares one creation method per product in the family. Each concrete factory implements all of them for one family, so the Budget factory can only produce budget products. `Presets.Build` receives the factory as an `IOnboardingPresetFactory` and never mentions `BudgetPresetFactory` or `ComfortPresetFactory`. It asks for both products and checks that they fit before returning the pair.

The demo loops over both families, saves each group and profile, and joins them:

```
foreach (var (name, factory) in families)
{
    OnboardingPreset preset = Presets.Build(factory);
    GroupView group = groups.CreateGroup(preset.Group);
    ProfileView profile = profiles.CreateProfile(preset.SuggestedProfile);
    GroupView joined = groups.JoinGroup(group.Id, profile.Id);
    ...
}
```

Both joins succeed, which shows that each family is internally consistent.

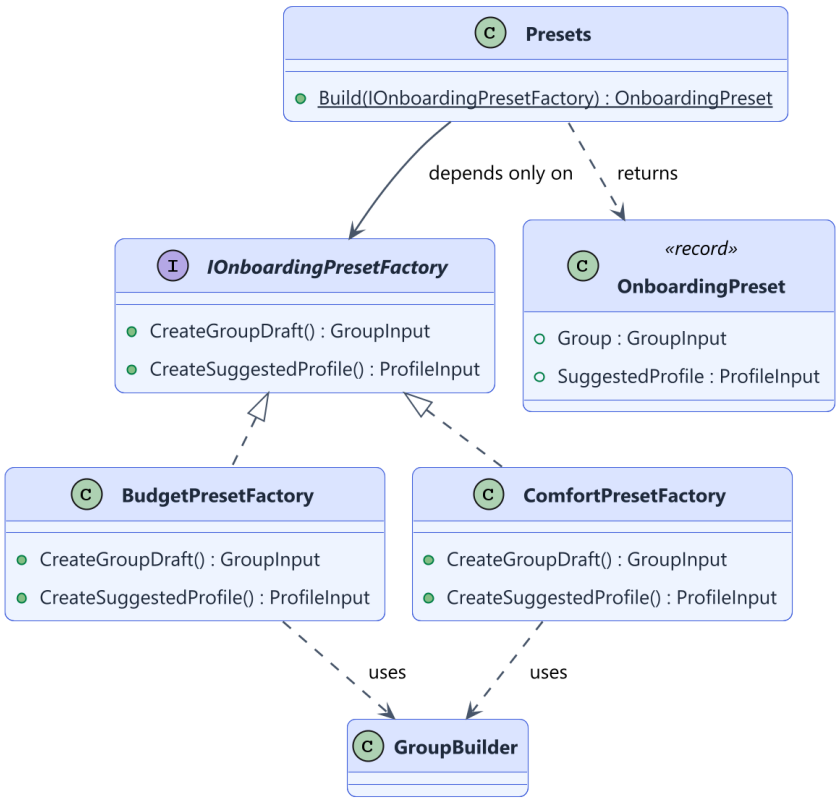


Figure 4.1 – Abstract Factory class diagram. The client depends on the interface only.

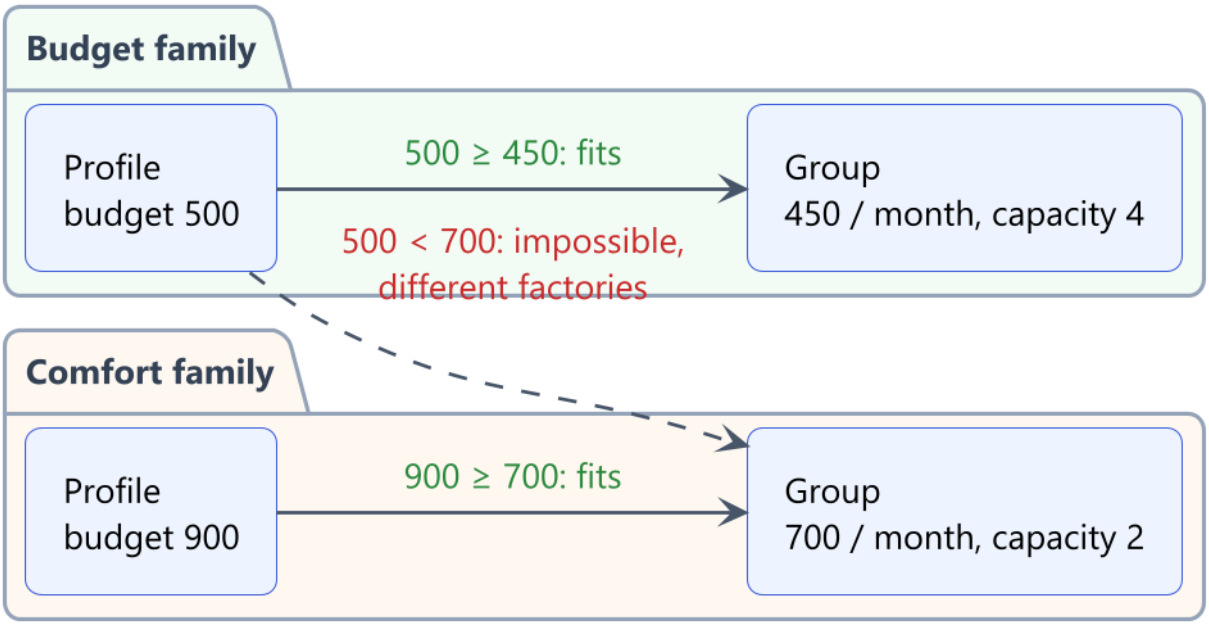


Figure 4.2 – Each factory produces a matching pair. A cross-family pair cannot be produced because one factory creates both products.

4.6 Why not something simpler

The alternative is two `switch` statements, one choosing the group and one choosing the profile. When a third style is added, both must be updated, and forgetting one creates exactly the mismatch above. With the factory, a new *Student* family is one new class, and `Presets.Build` and the demo loop do not

change. That is the Open/Closed Principle in practice, and the client depending on the interface instead of concrete classes is Dependency Inversion.

4.7 Trade-offs

Adding a new **family** is cheap: one class. Adding a new **product** to every family (for example, a suggested apartment) is expensive: the interface and every factory must change. Abstract Factory is a good choice when families change more often than the kinds of products in them.

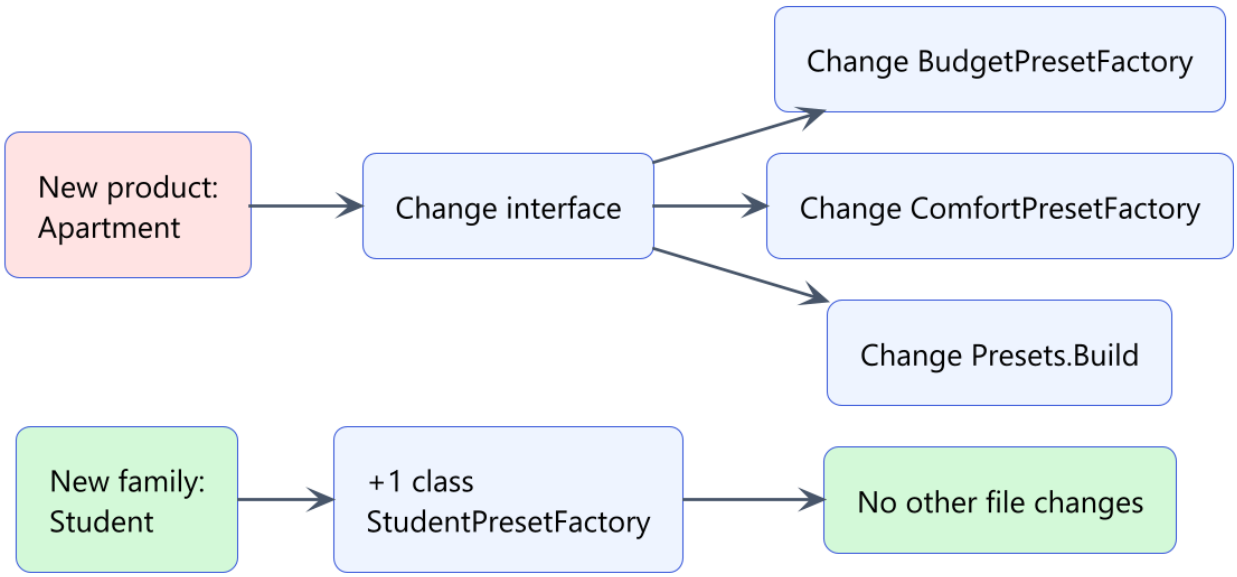


Figure 4.3 – The Abstract Factory trade-off: families are cheap to add, product types are not.

5 PROTOTYPE

5.1 The idea in plain words

Think of a form you already filled in last year. Instead of filling a blank one again, you photocopy it, then cross out your old signature and date. You keep what is still useful and remove what belonged only to the original.

A Prototype creates a new object by **copying an existing one**, then resetting what must be unique. In Domi: copy a group's cost and capacity, give it a new name, and leave out the ID and the members.

To copy correctly you need to know what a **reference** is. A variable of a class type does not hold the object; it holds the object's address in memory.

```
Group a = new Group(...);  // one object in memory, a holds its address
Group b = a;               // b gets the same address: still ONE object
b.AddMember(x);            // a sees x too, because it is the same object
```

So `Group b = a;` is not a copy. A real copy needs a new object, and inside it, new copies of any lists.

5.2 Problem

A user wants ”a group like this one”: the same cost and capacity, a new name, and nobody in it yet. Copying the fields from outside forces the caller to know every field, and it breaks when a field is added. Assigning the object (`Group copy = source;`) creates no copy at all, just a second reference to the same group.

5.3 Where it fits in Domi

A *Duplicate group* action, or reusing last semester's group settings for a new group.

5.4 Code

```
public sealed class GroupPrototype
{
    private readonly Group _template;

    public GroupPrototype(Group source)
    {
        _template = source.Copy(source.Id, source.Name, source.MemberProfileIds);
    }

    public Group CloneAsDraft(string name) => _template.Copy(id: "", name: name,
    memberIds: []);
}
```

The copy itself lives inside `Group`, because only `Group` can reach its private member list:

```
internal Group Copy(string id, string name, IEnumerable<string> memberIds)
{
```

```
    Group copy = new(id, new GroupInput(name, MaxMonthlyCost, Capacity));
    copy._memberProfileIds.AddRange(memberIds);
    return copy;
}
```

Usage:

```
Group source = groups.Get(budgetGroupId);
GroupPrototype prototype = new(source);
Group draft = prototype.CloneAsDraft("Budget shared home copy");
GroupView copy = groups.CreateGroup(draft.ToInput());
```

5.5 How it works

The constructor takes a **snapshot** of the source. `Copy` creates a brand-new `Group` through the normal constructor (so validation runs) and fills a **new** list with the source's member IDs. Later changes to the source cannot affect the snapshot.

`CloneAsDraft` copies the snapshot again, keeping cost and capacity, but with an empty ID, a new name and no members. The service then saves the draft and assigns a fresh ID.

Two things are deliberately not copied:

- **The ID.** It is the identity of the original record. Two records with the same ID would be the same group to the store, which is a bug;
- **The members.** They joined the original group. A new group starts empty;

5.6 Shallow copy and deep copy

This is the central idea of Prototype. A field of type `List<string>` does not hold the list; it holds a **reference** to it. A **shallow copy** copies field values, so the copy receives the same reference, and both groups point to one list. A **deep copy** creates a new list.

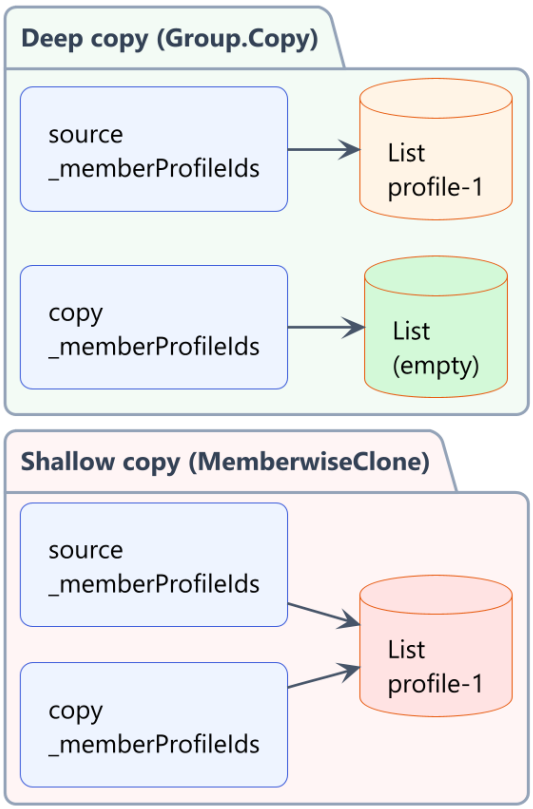


Figure 5.1 – With a shallow copy, adding a member to the copy also adds it to the source, because there is only one list. The deep copy gives each group its own list.

C# offers `object.MemberwiseClone()`, which performs a shallow copy, and the `ICloneable` interface, which does not say whether its `Clone()` is shallow or deep. Both are reasons this lab writes the copy explicitly.

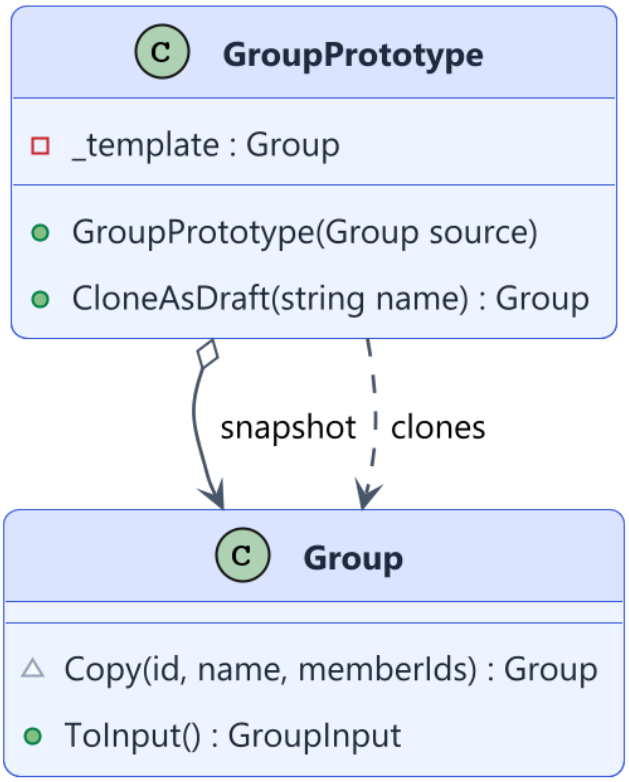


Figure 5.2 – Prototype class diagram.

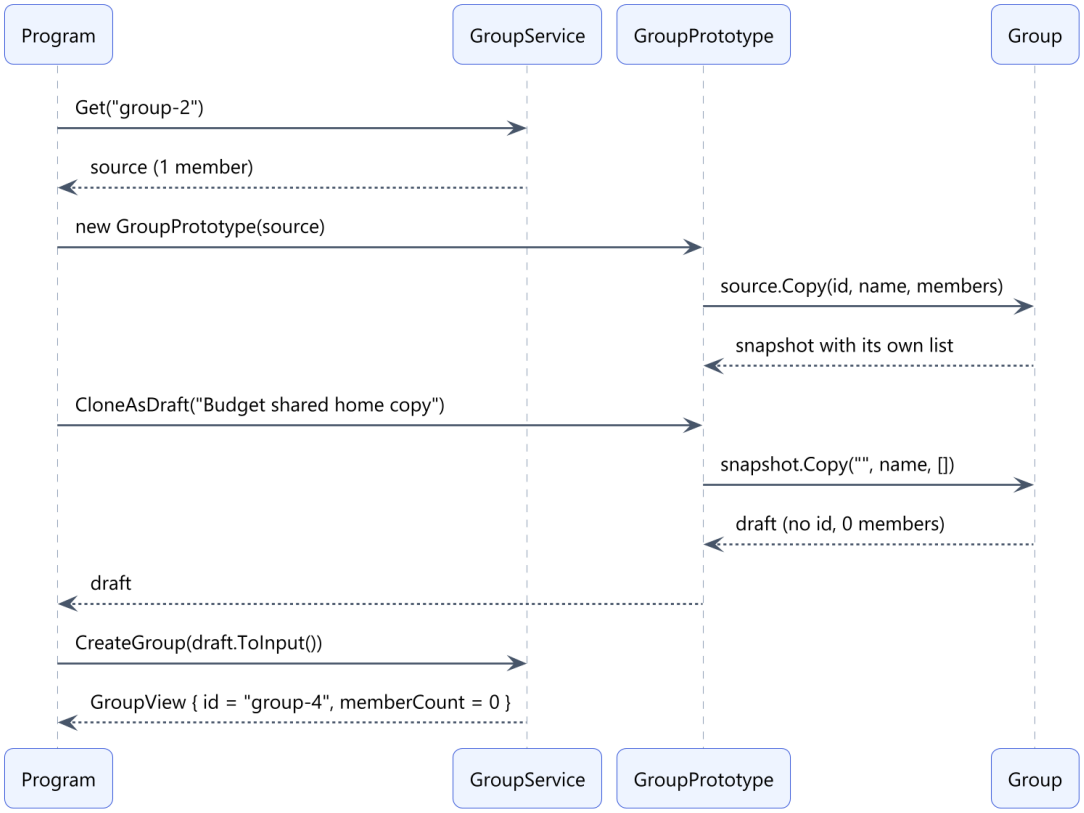


Figure 5.3 – Prototype sequence. The copy receives a new ID from the service.

5.7 Trade-offs

Every new field forces a decision: copy it, deep-copy it, or reset it. A forgotten field produces a wrong copy without any error. Objects with references to other objects (graphs, cycles) make deep copying harder.

6 SINGLETON

6.1 First, what `static` means

A normal field belongs to **each object**. A `static` field belongs to the **class itself**, so there is exactly one of it in the whole program.

Think of a classroom. Every student has their own notebook (a normal field), and there is one whiteboard on the wall that everyone shares (a static field).

```
class Student
{
    public string Name;            // notebook: each student has their own
    public static string Board;    // whiteboard: one for all students
}
```

With 100 `Student` objects there are 100 `Name` values but still only one `Board`, read through the class name: `Student.Board`.

Static does **not** mean ”cannot change”. That is `const` or `readonly`. Static means ”one shared copy”.

6.2 The idea in plain words

A Singleton is a class that allows **only one object of itself to ever exist**, and gives everyone the same way to reach it.

Think of a school bell. The school has one bell, and every classroom hears the same one. If each classroom had its own bell ringing at a slightly different time, nobody would know when the break starts.

A static field is one shared *variable*. A Singleton uses one static field to hold one shared *object*, and blocks everyone from creating a second one.

6.3 Problem it solves

Some things must exist only once in a program, because two copies would disagree or waste something scarce:

- **Configuration.** If two config objects were loaded at different times, one part of the program could use the old database address and another part the new one;
- **A logger.** Two loggers writing to the same file at once would mix their lines together;
- **A single device.** One printer queue or one connection to a hardware sensor. Two objects controlling it would send conflicting commands;

6.4 Where it fits in Domi

Domi's catalog defaults (default capacity 3, default cost 500) are the same for the whole application. Every part of the program must see the same values. The group creators in section 8 read them through

`CatalogDefaults.Instance`.

6.5 Where Singleton is used in real software

Use	Why one instance
Application configuration	Everyone must read the same settings
Logger	All log lines go through one writer, in order
Cache	One shared memory of results, so work is not repeated
Connection pool	One pool object manages all database connections (section 9)
`HttpClient` in .NET	Microsoft recommends sharing one instance; creating many runs out of network sockets

6.6 Where Singleton should NOT be used

- **User data: groups, profiles, a shopping cart, the current user.** There are many of them, one per person. A Singleton `CurrentUser` on a server would make every visitor the same user;
- **Anything that changes often and is read by many parts of the program.** One shared object that everyone can modify means any part of the code can break any other part, and bugs are hard to trace;
- **Things you need to replace in tests.** Code that calls `CatalogDefaults.Instance` directly always gets the real object. A test cannot give it fake values;
- **Just to avoid passing a parameter.** ”It's easier to reach from everywhere” is not a reason. That turns it into a global variable;

6.7 Code

```
public sealed class CatalogDefaults
{
    private static readonly Lazy<CatalogDefaults> _instance = new(() => new
   CatalogDefaults());

    public static CatalogDefaults Instance => _instance.Value;

    private CatalogDefaults() { }

    public int DefaultCapacity { get; } = 3;
    public int DefaultMaxMonthlyCost { get; } = 500;
}
```

6.8 How it works

Three parts make it a Singleton, and each one answers a question:

- **How do we stop people creating a second one?** `private CatalogDefaults()`: no code outside the class can write `new CatalogDefaults();`
- **Where is the one object kept?** `static ... _instance`: the field belongs to the class, so there is one for the whole program (the whiteboard);
- **How does everyone reach it?** `public static Instance`: anyone writes `CatalogDefaults.Instance` and gets that same object;

Two details make it safe:

- `Lazy<T>`: creates the object the first time `.Value` is read and guarantees, by default, that this happens once even when several threads ask at the same moment [2];
- `sealed`: no subclass can add a second constructor path;
- Properties have only `get`, so the shared object cannot be changed;

The demo prints `Singleton same instance: True` by comparing two reads with `ReferenceEquals`

.

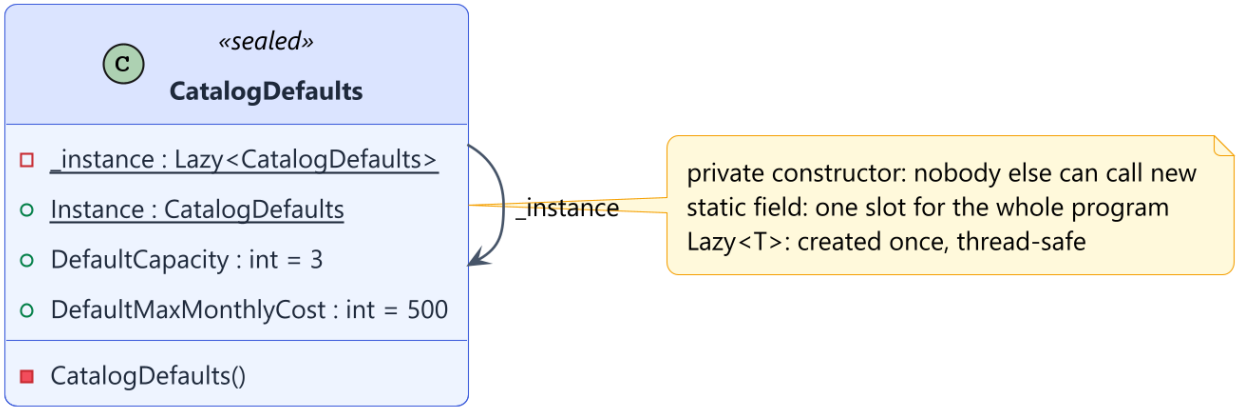


Figure 6.1 – Singleton class diagram. The class holds its only instance.

6.9 Why thread safety matters

The naive version checks and creates in two steps:

```
if (_instance == null) _instance = new Config();
return _instance;
```

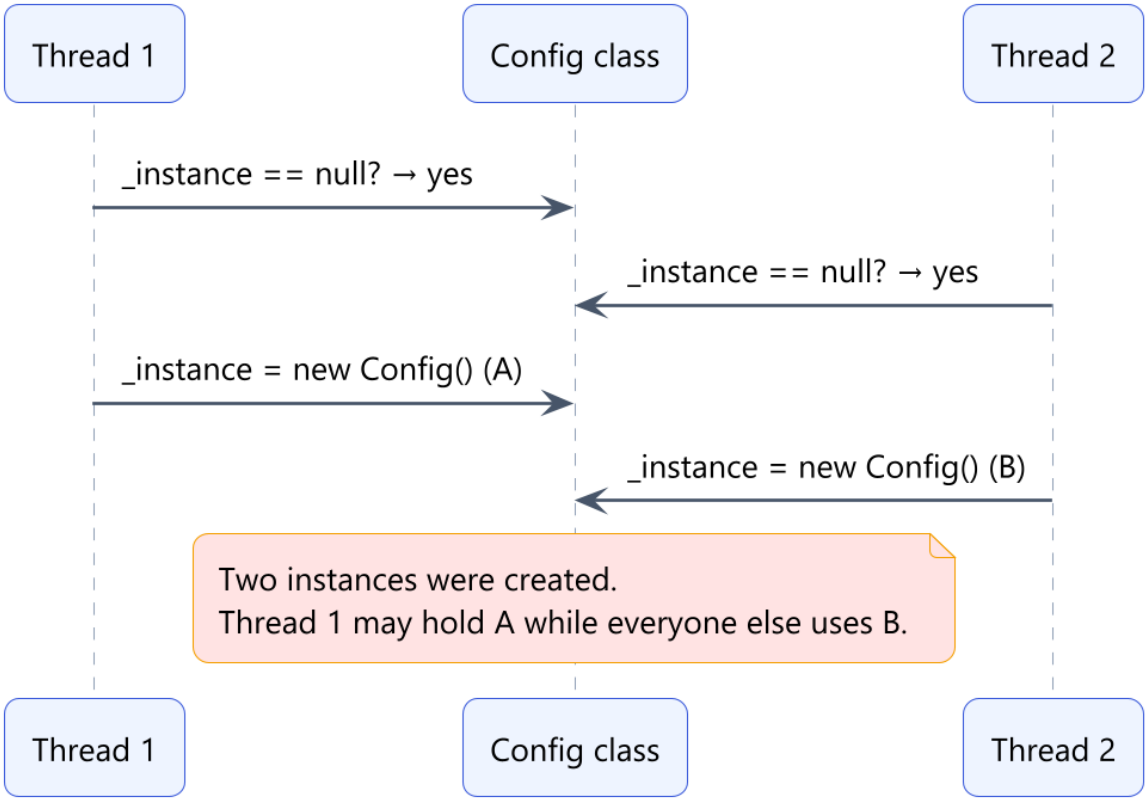


Figure 6.2 – A race in the naive Singleton. Both threads pass the null check before either assigns.

`Lazy<T>` removes the race by locking internally during the first creation.

6.10 Disadvantages: why Singleton is often criticised

A Singleton is global state. Any class can call `CatalogDefaults.Instance` without declaring it in its constructor, so its dependencies are hidden, which goes against Dependency Inversion. In a unit test the instance cannot be replaced with a fake, and mutable state in a Singleton leaks from one test to the next. Modern .NET applications usually register a service with a single lifetime in the dependency injection container instead: [3]

```
services.AddSingleton<ICatalogDefaults, CatalogDefaults>();
```

There is still one instance, but classes receive it through their constructors and tests can pass a different one. In Domi the Singleton is limited to immutable defaults. Groups and profiles are per-user data with many instances, so they are never Singletons.

Short answer for the defense: ”A Singleton guarantees one object and one global access point. I use it for things that must be shared and the same everywhere, like configuration or a logger; in Domi, the catalog defaults. I don't use it for user data like groups or profiles, because there are many of them, and I avoid it for anything that changes, because it's global state that hides dependencies and is hard to test. In modern C#, the usual way is to register a service as a singleton in dependency injection.”

7 FACTORY METHOD

7.1 The idea in plain words

Think of a pizza chain. Every shop follows the same steps: **make the pizza, bake it, box it**. Only the first step differs: the Italian shop makes thin crust, the American shop thick crust. Head office writes the steps once, and each shop only says how *it* makes the pizza.

In code, the parent class writes the steps, and the ”make” step is an **abstract** method (no body). Each child class **overrides** that one method, which means it writes its own version of it. That one creating method is the **factory method**. The other steps are not; they are the fixed workflow around it.

7.2 Problem, step by step

Start: creation with `if`. A service creates groups itself:

```
public Group Create(string type)
{
    if (type == "student") return new Group("Student flat", 300, 5);
    if (type == "quiet")   return new Group("Quiet home", 500, 2);
    throw new Exception("unknown type");
}
```

To add ”family” groups, you must **open this method and edit it**. Every edit can break code that already works. This breaks the Open/Closed Principle: add new behavior by adding code, not by editing old code.

First fix: Simple Factory. Move the `if` chain into a separate `GroupFactory.Create(type)`. The service is cleaner, but a new type still means editing the `if` chain. The problem moved; it did not disappear. Simple Factory is a common idiom but not one of the official patterns.

Real fix: Factory Method. Replace the `if` with inheritance. The parent keeps the workflow; each type of group is a child class. A new type is a new class, and no existing code changes.

7.3 Where it fits in Domi

Creating a group with its founder always means: create the group, save the founder's profile, join them. Only the kind of group differs: a student flat (cheap, five people) or a quiet home (two people).

7.4 Code

```
public abstract class GroupCreator
{
    public GroupView CreateAndJoin(GroupService groups, ProfileService profiles,
   ProfileInput founder)
    {
        GroupInput draft = CreateGroupDraft();
        GroupView group = groups.CreateGroup(draft);
        ProfileView profile = profiles.CreateProfile(founder);
```

```
        return groups.JoinGroup(group.Id, profile.Id);
    }

    protected abstract GroupInput CreateGroupDraft();
}

public sealed class StudentGroupCreator : GroupCreator
{
    protected override GroupInput CreateGroupDraft() => new GroupBuilder()
        .WithName("Student flat")
        .WithMaxMonthlyCost(300)
        .WithCapacity(CatalogDefaults.Instance.DefaultCapacity + 2)
        .Build();
}

public sealed class QuietGroupCreator : GroupCreator
{
    protected override GroupInput CreateGroupDraft() => new GroupBuilder()
        .WithName("Quiet home")
        .WithMaxMonthlyCost(CatalogDefaults.Instance.DefaultMaxMonthlyCost)
        .WithCapacity(2)
        .Build();
}
```

7.5 How it works

`CreateAndJoin` is a normal method in the base class and contains the fixed workflow. At its first step it calls `CreateGroupDraft()`, which is `abstract`: the base class does not know what it returns. When the program calls `new StudentGroupCreator().CreateAndJoin(...)`, the runtime dispatches `CreateGroupDraft()` to the `StudentGroupCreator` override. This is **polymorphism**: the same call in the base class runs different code depending on the actual object.

`CreateGroupDraft` is the **factory method**. It is `protected` because only the workflow should call it.

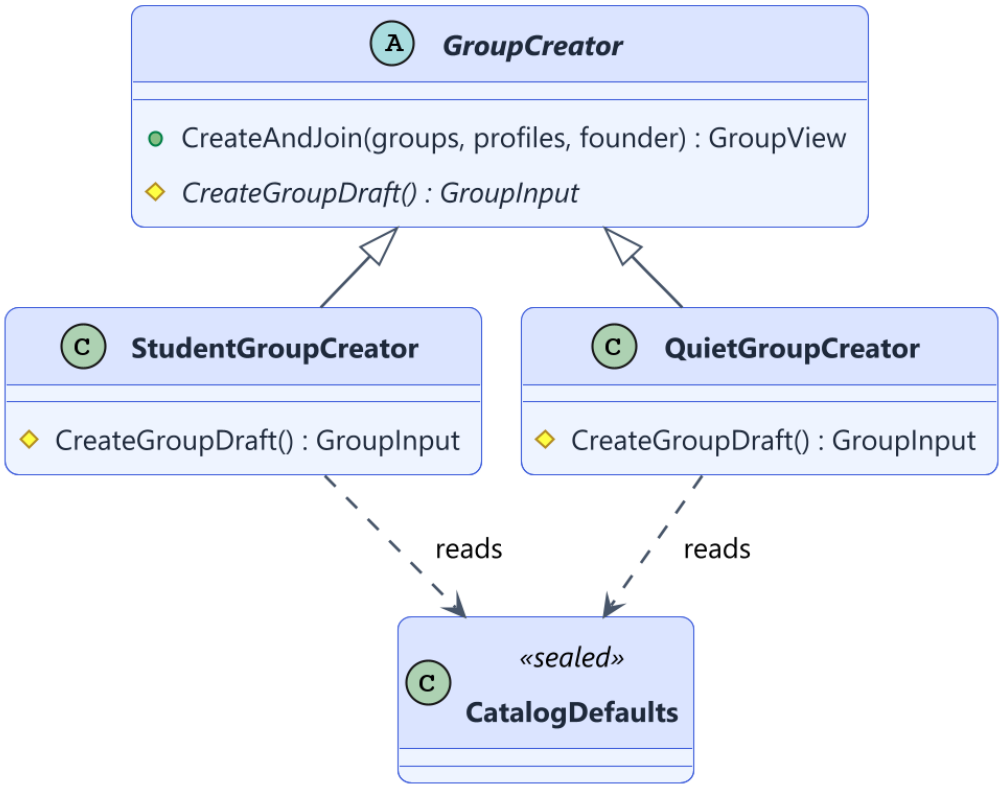


Figure 7.1 – Factory Method class diagram. The subclasses override only the creation step.

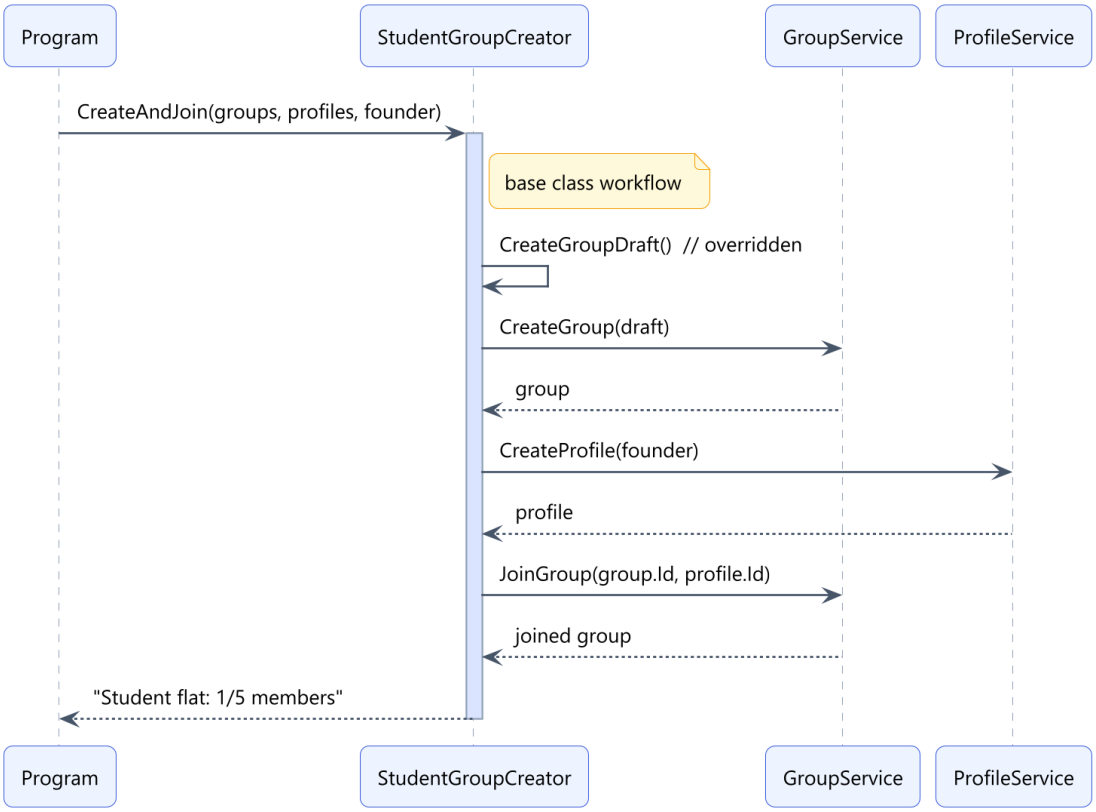


Figure 7.2 – Factory Method sequence. The workflow is inherited; one step is overridden.

7.6 Factory Method and Abstract Factory compared

Both create objects through an abstraction, which is why they are easy to confuse. They differ in what they create and in which OOP mechanism varies the result.

	Factory Method	Abstract Factory
Creates	one product	a family of related products
Varies through	inheritance: a subclass overrides one method	composition: the client receives a factory object
The creating code lives in	a method inside the class that uses the product	a separate factory object
In Domi	`GroupCreator.CreateGroupDraft()`	`IOnboardingPresetFactory`

Each method of an Abstract Factory is often implemented as a factory method, so the two patterns frequently appear together.

In the analogies: Factory Method is the pizza chain, where the steps are the same and each shop makes its own crust. Abstract Factory is the furniture style, where one choice gives a whole matching set.

Short answer for the defense: ”Factory Method uses inheritance: a child overrides one method to decide which single object is created inside a fixed workflow. Abstract Factory uses an interface: I pass in one factory object that creates a whole family of objects that match each other.”

A **Simple Factory**, a single static method with a `switch` that returns different types, is a common idiom but not one of the Gang of Four patterns. It centralises creation but must be edited for every new type, which Factory Method avoids.

7.7 Trade-offs

Each variant needs a subclass. When variants differ only in data, a parameter or a Builder is simpler than a class hierarchy.

8 OBJECT POOL

8.1 The idea in plain words

Think of a library. Instead of buying a new book every time someone wants to read, the library keeps its books. You **borrow** one, read it, and **return** it, and the next person borrows the same book.

An Object Pool keeps finished objects on a shelf. Code **acquires** (borrows) one, uses it, and **releases** (returns) it. If the shelf is empty, the pool creates a new object.

8.2 Problem

Some objects are expensive to create or exist in limited numbers: database connections, threads, network sockets, large buffers. Opening a database connection means reaching the server over the network and logging in, which takes far longer than running a query. A database also accepts only a fixed number of connections at once. Creating and destroying a connection for every request is slow and can hit that limit.

8.3 Where it fits in Domi

Domi builds short text summaries of groups. The pool reuses `StringBuilder` buffers between summaries. In a production backend the same idea is applied to database connections; ADO.NET keeps a connection pool for each connection string, and `ArrayPool<T>.Shared` does the same for arrays [4].

8.4 Code

```
public sealed class SummaryBuilderPool(int maxIdle)
{
    private readonly Stack<StringBuilder> _idle = new();

    public int Created { get; private set; }

    public StringBuilder Acquire()
    {
        if (_idle.Count > 0) return _idle.Pop();
        Created++;
        return new StringBuilder();
    }

    public void Release(StringBuilder builder)
    {
        builder.Clear();
        if (_idle.Count < maxIdle) _idle.Push(builder);
    }

    public string Summarize(GroupView group)
    {
        StringBuilder sb = Acquire();
        try
        {
            return sb.Append(group.Name).Append(": ")
                .Append(group.MemberCount).Append('/').Append(group.Capacity)
```

```
                .Append(" members, ").Append(group.MaxMonthlyCost).Append(" per month
    ")
                .ToString();
        }
        finally
        {
            Release(sb);
        }
    }
}
```

8.5 How it works

The pool keeps idle objects on a stack.

- **Acquire** returns an idle builder if there is one. Otherwise it creates a new one and counts it in `Created`;
- **Release** clears the builder and puts it back, unless the pool already holds `maxIdle` idle builders. In that case the builder is dropped and the garbage collector frees it;
- **Summarize** shows the correct usage: acquire, use, and release in `finally`, so the builder returns to the pool even when an exception is thrown;

The demo creates two summaries with one pool and prints `Pool created 1 StringBuilder(s) for 2 summaries`: the second summary reused the first builder.

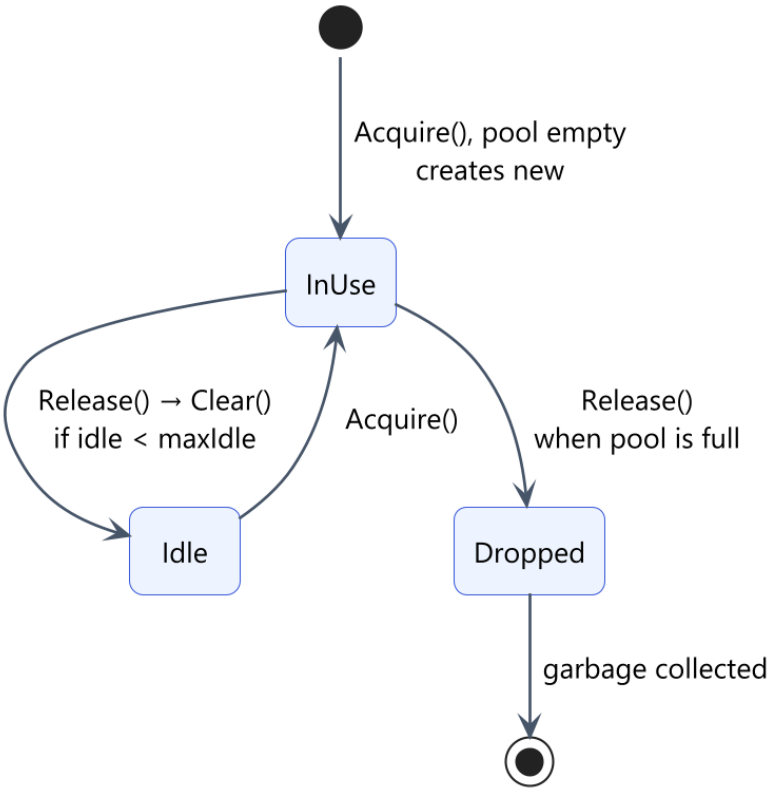


Figure 8.1 – Life cycle of a pooled object.

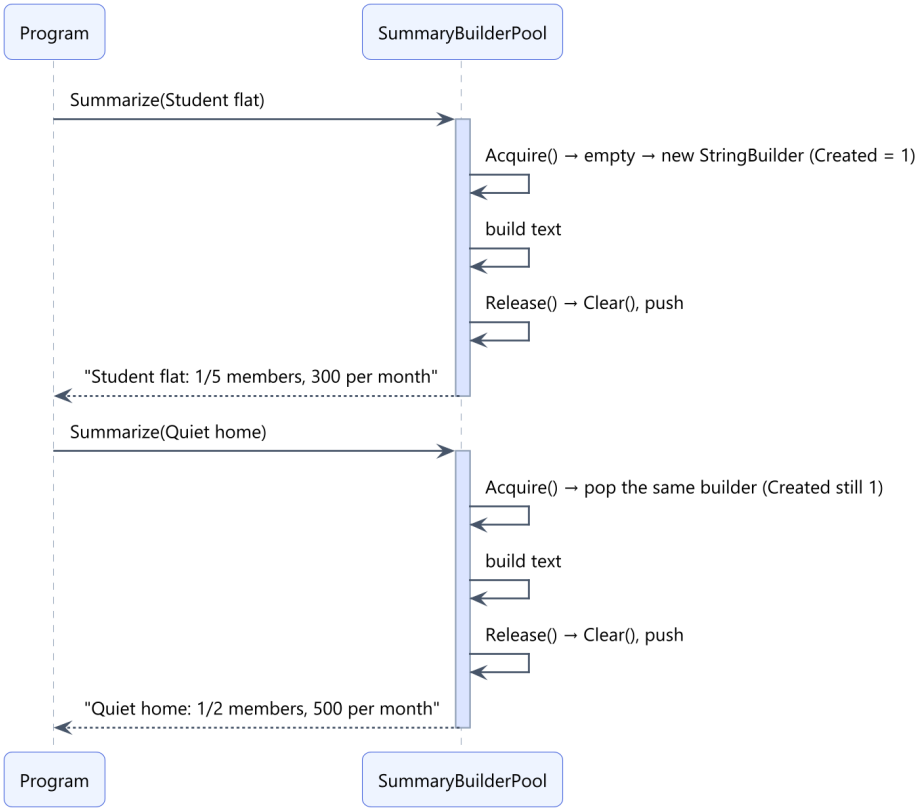


Figure 8.2 – Two summaries, one builder.

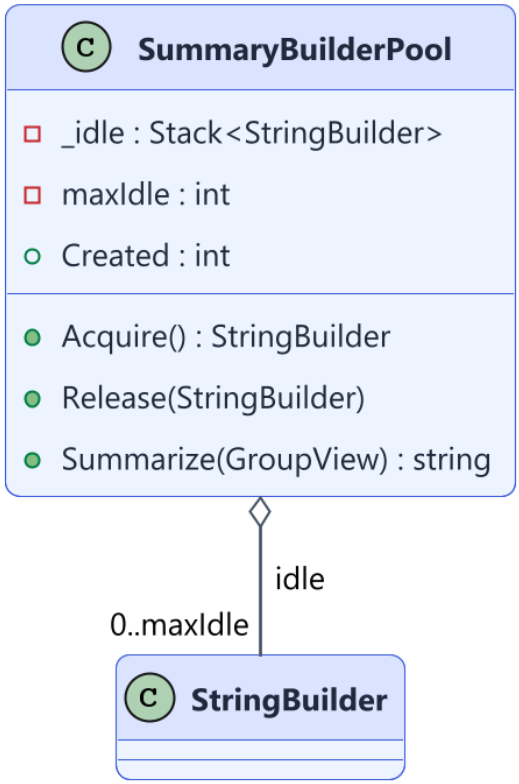


Figure 8.3 – Object Pool class diagram.

8.6 The two rules of pooling

1. **Reset on release.** Without `Clear()`, the next user would receive a builder that still contains the previous summary. With database connections the same mistake leaks an open transaction or another user's session settings;
2. **Always release.** If a user acquires an object and an exception skips the release, the object never returns, like a library book that is never brought back. A pool without a limit keeps creating new objects and the benefit is lost. A pool with a hard limit, such as 100 database connections, is worse: after 100 lost connections, the 101st request **waits forever** and the application hangs. `try/finally` (or `using` with a disposable wrapper) prevents this, because `finally` runs even after an error;

`maxIdle` caps how much the pool holds when demand drops, so a burst of traffic does not leave memory reserved forever.

8.7 Trade-offs

Pooling pays off only when creation is expensive compared with the bookkeeping. A small `StringBuilder` is cheap, and the .NET garbage collector handles short-lived objects efficiently, so in Domi the pool is mainly a clear demonstration of the mechanism. This pool is also not thread-safe; a shared pool used by several threads would need `ConcurrentBag<T>` or a lock.

9 HOW THE PATTERNS WORK TOGETHER

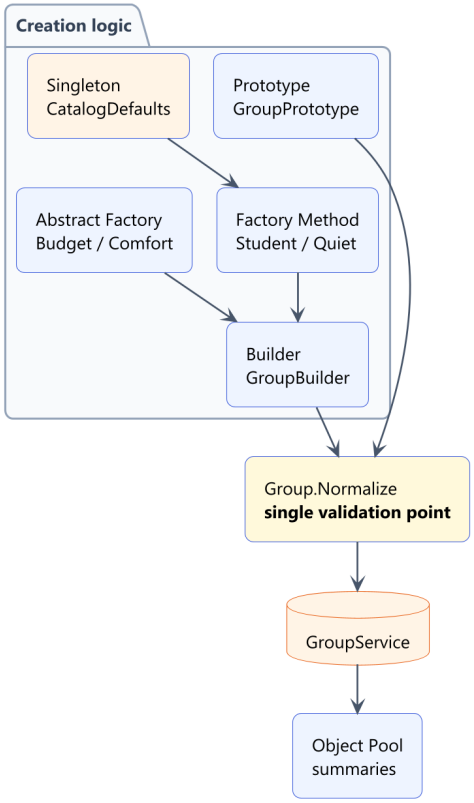


Figure 9.1 – Every creation path ends in `Group.Normalize` before the service stores the group.

The patterns combine instead of competing. Both factories use the Builder to assemble their group drafts, the Factory Method creators read defaults from the Singleton, and the Prototype goes through the same constructor as everything else. Because every path converges on one validation method, adding a new creation path cannot weaken the rules.

9.1 Connection to SOLID (Lab 0)

Principle	Where this lab applies it
Single Responsibility	The Builder assembles, `Normalize` validates, the services store. The Prototype only copies.
Open/Closed	A new onboarding family or a new group creator is a new class; existing code does not change.
Liskov Substitution	Any `IOnboardingPresetFactory` or `GroupCreator` subclass can replace another without breaking the caller.
Interface Segregation	`IOnboardingPresetFactory` contains only the two methods onboarding needs.

Principle	Where this lab applies it
Dependency Inversion	`Presets.Build` depends on `IOnboardingPresetFactory`, not on concrete factories. The Singleton section explains where it conflicts with this principle and how DI fixes it.

9.2 Choosing a pattern

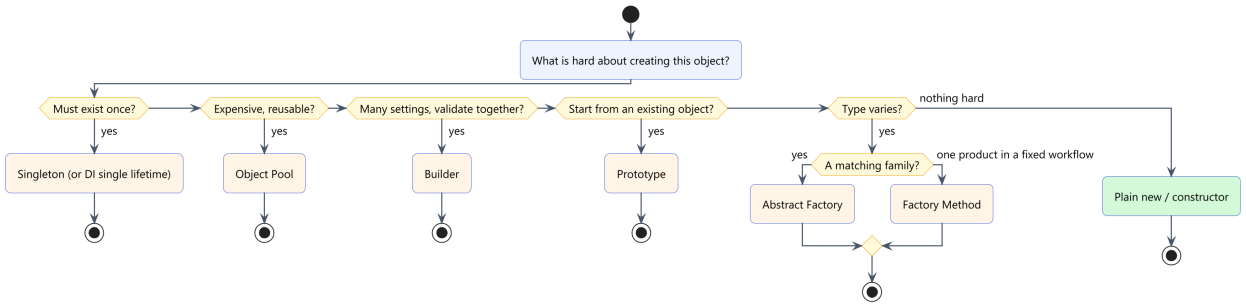


Figure 9.2 – A decision guide. ”Nothing hard” is a valid answer: patterns are for problems that exist.

10 RESULTS

Running `dotnet run`:

```
{
  "builderGroup": { "id": "group-1", "name": "Builder example home", "maxMonthlyCost
   ": 650, "capacity": 3, "memberCount": 0 },
  "presetGroups": [
    {
      "family": "budget",
      "group": { "id": "group-2", "name": "Budget shared home", "maxMonthlyCost":
   450, "capacity": 4, "memberCount": 1 },
      "suggestedProfile": { "id": "profile-1", "name": "Budget renter" }
    },
    {
      "family": "comfort",
      "group": { "id": "group-3", "name": "Comfort shared home", "maxMonthlyCost":
   700, "capacity": 2, "memberCount": 1 },
      "suggestedProfile": { "id": "profile-2", "name": "Comfort renter" }
    }
  ],
  "prototype": {
    "sourceGroupId": "group-2",
    "sourceMemberCount": 1,
    "copy": { "id": "group-4", "name": "Budget shared home copy", "maxMonthlyCost":
   450, "capacity": 4, "memberCount": 0 }
  }
}
```

Output	What it shows
`builderGroup` is `group-1` with the three values set	The Builder produced a valid input that the service saved.
Both preset groups have `memberCount: 1`	Each factory produced a matching pair, so the suggested renter could join.
The prototype source has 1 member, the copy has 0 and a new ID `group-4`	The clone copied the settings, not the identity or the members, and the source was not affected.

Running `dotnet run -- --extra`:

```
Singleton same instance: True
Student flat: 1/5 members, 300 per month
Quiet home: 1/2 members, 500 per month
Pool created 1 StringBuilder(s) for 2 summaries
```

Output	What it shows
`same instance: True`	Two reads of `CatalogDefaults.Instance` return the same object.
Student 1/5 at 300, Quiet 1/2 at 500	One inherited workflow, two different overridden creation steps. Capacity 5 and cost 500 come from the Singleton's defaults.
`Created 1 ... for 2 summaries`	The second summary reused the pooled builder.

10.1 Project structure

```
lab-01/csharp/
  DomiLab01.csproj   .NET 8 console project
  Domain.cs          Group, Profile, inputs, views, validation
  Services.cs        GroupService, ProfileService
  Patterns.cs        Builder, Abstract Factory, Prototype
  ExtraPatterns.cs   Singleton, Factory Method, Object Pool
  Program.cs         Demo for both runs
```

CONCLUSIONS

Creating an object becomes a design problem when the caller should not depend on the concrete class, when construction has rules that must hold together, or when the number of instances matters. This lab applied the six creational patterns to those situations in Domi.

The **Builder** made group creation readable and validated it once at the end. The **Abstract Factory** guaranteed that an onboarding group and its suggested renter always match, and showed its trade-off: new families are cheap, new product types are not. The **Prototype** reused a group's settings while deliberately leaving out its identity and members, which made the difference between a shallow and a deep copy concrete. The **Singleton** gave one shared set of defaults, together with the reasons it is usually replaced by a dependency injection lifetime. The **Factory Method** kept one workflow and let subclasses choose the group, which is the inheritance-based counterpart to Abstract Factory. The **Object Pool** reused buffers and showed the two rules that make pooling safe: reset on release and always release.

The most useful lesson is that the patterns serve the SOLID principles from Lab 0. They move the decision of *what to create* away from the code that *uses* the object, so that new kinds of groups, families and copies can be added without editing the code that already works.

REFERENCES

1. E. Gamma, R. Helm, R. Johnson, J. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.

2. Microsoft. *Lazy<T> Class*. Available online: `https://learn.microsoft.com/dotnet/api/system.lazy-1` (accessed on 3 November 2026).

3. Microsoft. *Dependency injection in .NET: service lifetimes*. Available online: `https://learn.microsoft.com/dotnet/core/extensions/dependency-injection` (accessed on 3 November 2026).

4. Microsoft. *ArrayPool<T> Class*. Available online: `https://learn.microsoft.com/dotnet/api/system.buffers.arraypool-1` (accessed on 3 November 2026).